

Assignment 12.3

M.koushik reddy
2103A52153

Task 1: Sorting Student Records for Placement Drive Scenario

```
import random
import string
import time
import copy

# -----
# Student Class
# -----

class Student:
    def __init__(self, name, roll, cgpa):
        self.name = name
        self.roll = roll
        self.cgpa = cgpa

    def __repr__(self):
        return f"{self.name} | {self.roll} | CGPA: {self.cgpa}"

# -----
# Generate Large Dataset
# -----

def generate_students(n):
    students = []
    for i in range(n):
        name = ''.join(random.choices(string.ascii_uppercase, k=5))
        roll = f"SRU{i:05d}"
        cgpa = round(random.uniform(5.0, 10.0), 2)
```

```

        students.append(Student(name, roll, cgpa))
    return students

# -----
# Quick Sort (Descending CGPA)
# -----
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr)//2].cgpa
    left = [x for x in arr if x.cgpa > pivot]
    middle = [x for x in arr if x.cgpa == pivot]
    right = [x for x in arr if x.cgpa < pivot]

    return quick_sort(left) + middle + quick_sort(right)

# -----
# Merge Sort (Descending CGPA)
# -----
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i].cgpa > right[j].cgpa:
            result.append(left[i])

```

```

        i += 1
    else:
        result.append(right[j])
        j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result

# -----
# Display Top 10 Students
# -----
def display_top_10(sorted_students):
    print("\nTop 10 Students Based on CGPA:\n")
    for student in sorted_students[:10]:
        print(student)

# -----
# Main Execution
# -----
if __name__ == "__main__":
    n = 50000    # Large dataset size
    students = generate_students(n)

    # Quick Sort Test
    students_qs = copy.deepcopy(students)
    start = time.time()
    sorted_qs = quick_sort(students_qs)
    end = time.time()
    quick_time = end - start
    print(f"Quick Sort Time: {quick_time:.4f} seconds")

    # Merge Sort Test
    students_ms = copy.deepcopy(students)
    start = time.time()
    sorted_ms = merge_sort(students_ms)
    end = time.time()

```

```

merge_time = end - start
print(f"Merge Sort Time: {merge_time:.4f} seconds")

# Show Top 10 (from Quick Sort result)
display_top_10(sorted_qs)

# Performance Comparison
print("\nPerformance Comparison:")
print(f"Quick Sort: {quick_time:.4f} sec")
print(f"Merge Sort: {merge_time:.4f} sec")

```

Output

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\koushik reddy\AppData\Local\Programs\Microsoft VS Code> & "C:\Users\koushik reddy\AppData\Local\Micro
Quick Sort Time: 0.1164 seconds
Merge Sort Time: 0.1651 seconds

Top 10 Students Based on CGPA:

Quick Sort Time: 0.1164 seconds
Merge Sort Time: 0.1651 seconds

Top 10 Students Based on CGPA:

Top 10 Students Based on CGPA:

Top 10 Students Based on CGPA:

Top 10 Students Based on CGPA:

PUHNL | SRU00691 | CGPA: 10.0
SQTWH | SRU01240 | CGPA: 10.0
PUHNL | SRU00691 | CGPA: 10.0
SQTWH | SRU01240 | CGPA: 10.0
SQTWH | SRU01240 | CGPA: 10.0
HCORN | SRU03541 | CGPA: 10.0
FAAGA | SRU04456 | CGPA: 10.0
QXPIR | SRU07008 | CGPA: 10.0
RTEPW | SRU08357 | CGPA: 10.0
MKGHL | SRU09289 | CGPA: 10.0
WBQEB | SRU09733 | CGPA: 10.0
VZWFP | SRU09819 | CGPA: 10.0
MHUGN | SRU09912 | CGPA: 10.0

Performance Comparison:
Quick Sort: 0.1164 sec
Merge Sort: 0.1651 sec
PS C:\Users\koushik reddy\AppData\Local\Programs\Microsoft VS Code>
PS C:\Users\koushik reddy\AppData\Local\Programs\Microsoft VS Code> 

```

Task 2

```
def bubble_sort(arr):
    n = len(arr)

    # Outer loop controls the number of passes
    # After each pass, the largest unsorted element
    # moves ("bubbles") to its correct position at the end
    for i in range(n):

        # Flag to check if any swap happens in this pass
        # If no swaps occur, array is already sorted
        swapped = False

        # Inner loop compares adjacent elements
        # We reduce range by 'i' because last 'i' elements
        # are already sorted after each pass
        for j in range(0, n - i - 1):

            # Compare adjacent elements
            # If current element is greater than next,
            # swap them to maintain ascending order
            if arr[j] > arr[j + 1]:

                # Swapping elements
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

                # Mark that a swap occurred
                swapped = True

        # If no elements were swapped in this pass,
        # the array is already sorted → terminate early
        if not swapped:
            break

    return arr
```

Output

```
PS C:\Users\koushik reddy\AppData\Local\Programs\Microsoft VS Code> & "C:\Users\koushik reddy\AppData\Local\Programs\Microsoft VS Code>  
Sorted Array: [11, 12, 22, 25, 34, 64, 90]  
PS C:\Users\koushik reddy\AppData\Local\Programs\Microsoft VS Code> & "C:\Users\koushik reddy\AppData\Local\Programs\Microsoft VS Code>  
Sorted Array: [11, 12, 22, 25, 34, 64, 90]  
PS C:\Users\koushik reddy\AppData\Local\Programs\Microsoft VS Code>
```

Task 3 Quick Sort and Merge Sort Comparison

```
import time
import sys

# Increase recursion limit for large lists
sys.setrecursionlimit(10**6)

# -----
# Quick Sort (Recursive)
# -----

def quick_sort(arr):
    """
    Performs Quick Sort using recursion.

    Parameters:
        arr (list): List of comparable elements.
```

Returns:

list: New sorted list (ascending order).

Algorithm:

- Select a pivot (middle element).
- Partition elements into:
 - left -> elements smaller than pivot
 - equal -> elements equal to pivot
 - right -> elements greater than pivot
- Recursively sort left and right.

"""

```
if len(arr) <= 1:  
    return arr
```

```
pivot = arr[len(arr) // 2]
```

```
left = [x for x in arr if x < pivot]  
middle = [x for x in arr if x == pivot]  
right = [x for x in arr if x > pivot]
```

```
return quick_sort(left) + middle + quick_sort(right)
```

```
# -----  
# Merge Sort (Recursive)  
# -----
```

```
def merge_sort(arr):
```

"""

Performs Merge Sort using recursion.

Parameters:

arr (list): List of comparable elements.

Returns:

list: Sorted list (ascending order).

Algorithm:

- Divide array into two halves.
- Recursively sort both halves.

```

        - Merge sorted halves.
    """
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    """
    Merges two sorted lists into one sorted list.
    """
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result

# -----
# Performance Comparison
# -----
def test_algorithms(data, label):
    print(f"\nTesting on {label} list (size = {len(data)})")

# Quick Sort

```



```

start = time.time()
quick_sort(data)
quick_time = time.time() - start

# Merge Sort
start = time.time()
merge_sort(data)
merge_time = time.time() - start

print(f"Quick Sort Time: {quick_time:.6f} sec")
print(f"Merge Sort Time: {merge_time:.6f} sec")

# -----
# Main Execution
# -----
if __name__ == "__main__":
    n = 10000

    random_list = [random.randint(1, 100000) for _ in range(n)]
    sorted_list = sorted(random_list)
    reverse_sorted_list = sorted(random_list, reverse=True)

    test_algorithms(random_list, "Random")
    test_algorithms(sorted_list, "Sorted")
    test_algorithms(reverse_sorted_list, "Reverse Sorted")

```

Output

```

Merge Sort Time: 0.016433 sec
PS C:\Users\koushik reddy\AppData\Local\Programs\Microsoft VS Code> & "C:\Users\koushik reddy\AppData\Local\Microso

Testing on Random list (size = 10000)
Quick Sort Time: 0.024119 sec
Merge Sort Time: 0.017687 sec

Testing on Sorted list (size = 10000)
Quick Sort Time: 0.021142 sec
Merge Sort Time: 0.011191 sec

Testing on Reverse Sorted list (size = 10000)
Quick Sort Time: 0.016563 sec
Merge Sort Time: 0.015425 sec
PS C:\Users\koushik reddy\AppData\Local\Programs\Microsoft VS Code>

```

Task 4

(Real-Time Application – Inventory Management System)

Product class

```
class Product:
    def __init__(self, product_id, name, price, quantity):
        self.product_id = product_id
        self.name = name
        self.price = price
        self.quantity = quantity

    def __repr__(self):
        return f"{self.product_id} | {self.name} | ₹{self.price} | Stock: {self.quantity}"
```

Product class

```
class Inventory:
    def __init__(self):
        # Hash tables for fast search
        self.products_by_id = {}
        self.products_by_name = {}

    def add_product(self, product):
        # O(1) insertion
        self.products_by_id[product.product_id] = product
        self.products_by_name[product.name.lower()] = product

    # -----
    # Searching
    # -----

    def search_by_id(self, product_id):
```

```

        """
        Search product using hash table lookup.
        Time Complexity: O(1) average
        """
        return self.products_by_id.get(product_id, "Product not
found")

    def search_by_name(self, name):
        """
        Search product by name using hash table.
        Time Complexity: O(1) average
        """
        return self.products_by_name.get(name.lower(), "Product not
found")

# -----
# Sorting
# -----
    def sort_by_price(self):
        """
        Sort products by price using Python's Timsort.
        Time Complexity: O(n log n)
        """
        return sorted(self.products_by_id.values(), key=lambda x:
x.price)

    def sort_by_quantity(self):
        """
        Sort products by stock quantity.
        Time Complexity: O(n log n)
        """
        return sorted(self.products_by_id.values(), key=lambda x:
x.quantity)

```

Example Usage

```

class Inventory:
    def __init__(self):
        # Hash tables for fast search

```

```

        self.products_by_id = {}
        self.products_by_name = {}

    def add_product(self, product):
        # O(1) insertion
        self.products_by_id[product.product_id] = product
        self.products_by_name[product.name.lower()] = product

# -----
# Searching
# -----

    def search_by_id(self, product_id):
        """
        Search product using hash table lookup.
        Time Complexity: O(1) average
        """
        return self.products_by_id.get(product_id, "Product not
found")

    def search_by_name(self, name):
        """
        Search product by name using hash table.
        Time Complexity: O(1) average
        """
        return self.products_by_name.get(name.lower(), "Product not
found")

# -----
# Sorting
# -----

    def sort_by_price(self):
        """
        Sort products by price using Python's Timsort.
        Time Complexity: O(n log n)
        """
        return sorted(self.products_by_id.values(), key=lambda x:
x.price)

    def sort_by_quantity(self):
        """

```

```

        Sort products by stock quantity.
        Time Complexity: O(n log n)
        """
        return sorted(self.products_by_id.values(), key=lambda x:
x.quantity)

```

Output

```

Search by ID: P101 | Laptop | ₹60000 | Stock: 15
Search by Name: P102 | Mouse | ₹500 | Stock: 150

```

Sorted by Price:

```

P102 | Mouse | ₹500 | Stock: 150
P103 | Keyboard | ₹1500 | Stock: 80
P104 | Monitor | ₹12000 | Stock: 25
P101 | Laptop | ₹60000 | Stock: 15

```

Sorted by Quantity:

```

P101 | Laptop | ₹60000 | Stock: 15
P104 | Monitor | ₹12000 | Stock: 25
P103 | Keyboard | ₹1500 | Stock: 80
P102 | Mouse | ₹500 | Stock: 150

```

Task 5 : Real-Time Stock Data Sorting & Searching

```

import random
import string
import time
import heapq

# -----
# Stock Class
# -----
class Stock:

```

```

def __init__(self, symbol, open_price, close_price):
    self.symbol = symbol
    self.open_price = open_price
    self.close_price = close_price
    self.change_percent = ((close_price - open_price) /
open_price) * 100

def __repr__(self):
    return f"{self.symbol} | Open: {self.open_price:.2f} |
Close: {self.close_price:.2f} | Change: {self.change_percent:.2f}%"

# -----
# Generate Random Stock Data
# -----
def generate_stocks(n):
    stocks = []
    for _ in range(n):
        symbol = ''.join(random.choices(string.ascii_uppercase,
k=4))

        open_price = random.uniform(100, 1000)
        close_price = random.uniform(100, 1000)
        stocks.append(Stock(symbol, open_price, close_price))
    return stocks

# -----
# Heap Sort (Based on % Change)
# -----
def heap_sort(stocks):
    """
    Sort stocks in descending order based on percentage change.
    Time Complexity: O(n log n)
    """
    heap = []

    for stock in stocks:
        # Using negative value for max-heap behavior
        heapq.heappush(heap, (-stock.change_percent, stock))

```

```

        sorted_stocks = []
        while heap:
            sorted_stocks.append(heapq.heappop(heap)[1])

        return sorted_stocks

# -----
# Hash Map Search
# -----
def build_hash_map(stocks):
    """
    Build dictionary for O(1) stock lookup.
    """
    stock_map = {}
    for stock in stocks:
        stock_map[stock.symbol] = stock
    return stock_map

def search_stock(stock_map, symbol):
    """
    Search stock instantly using hash map.
    Time Complexity: O(1) average
    """
    return stock_map.get(symbol, "Stock not found")

# -----
# Performance Comparison
# -----
if __name__ == "__main__":
    n = 20000
    stocks = generate_stocks(n)

    # Build Hash Map
    stock_map = build_hash_map(stocks)

    # -----
    # Heap Sort Timing

```

```

# -----
start = time.time()
heap_sorted = heap_sort(stocks)
heap_time = time.time() - start

# -----
# Python Built-in Sort Timing
# -----
start = time.time()
built_in_sorted = sorted(stocks, key=lambda x: x.change_percent,
reverse=True)
builtin_time = time.time() - start

# -----
# Search Timing
# -----
random_symbol = stocks[n//2].symbol

start = time.time()
result = search_stock(stock_map, random_symbol)
search_time = time.time() - start

print("Heap Sort Time:", heap_time)
print("Built-in Sort Time:", builtin_time)
print("Search Time (Hash Map):", search_time)

print("\nTop 5 Gainers:")
for stock in heap_sorted[:5]:
    print(stock)

print("\nSearch Result:")
print(result)

```

Output


```
Heap Sort Time: 0.01975107192993164
Built-in Sort Time: 0.011782169342041016
Search Time (Hash Map): 0.0
```

Top 5 Gainers:

SLMX		Open: 100.73		Close: 990.07		Change: 882.92%
IDMS		Open: 101.21		Close: 981.40		Change: 869.65%
ZCWB		Open: 100.09		Close: 956.35		Change: 855.52%
VRMC		Open: 102.38		Close: 976.35		Change: 853.61%
PTHI		Open: 103.46		Close: 984.26		Change: 851.37%

Search Result:

HBDE		Open: 284.56		Close: 815.10		Change: 186.44%
------	--	--------------	--	---------------	--	-----------------

```
PS C:\Users\koushik reddy\AppData\Local\Programs\Microsoft VS Code> █
```